

- **Summary:**

1. **The structure and logic of the Component:**

Counter - A simple counter that lets users:

- Increment the value
 - Decrement the value
 - Reset it to zero
-
- Utilizes useState to manage the count.
 - Used onClick events on buttons update the state.
 - Uses styled components for modular, scoped CSS.
 - UI elements include Increment, Decrement, Reset buttons and Count display

A dynamic to-do list where users can:

- Add a new task
 - Mark a task as complete/incomplete (toggle)
 - Remove a task
 - Search/filter tasks in real-time
-
- Uses useReducer () to manage a list of tasks.
 - Uses useState () for input field and search filter.
 - Add: Adds a new task with a unique ID.
 - Toggle: Marks task as complete/incomplete.
 - Remove: Deletes task from the list.
 - A search input dynamically filters displayed tasks.
 - Styled components used for input, buttons, task boxes.
 - Completed tasks have a green background, upon clicking on the task name.

2. **React App Tests**

Contains basic tests using **Jest** and **React Testing Library**:

- Counter.test.js: Verifies increment, decrement, and reset functionality.
- TodoList.test.js: Validates adding, toggling (completion), and removing tasks

3. **React App Design**

Details the UI styling:

- Clean, modular CSS via styled components.
- Responsive design using media queries.
- Accessibility with large buttons and clear status cues (e.g., green background for completed tasks).

- **Design Decisions**

- Used functional components and hooks (useState, useReducer) to demonstrate modern React practices.
- Chose styled-components to keep styling scoped and maintainable.
- Structured the app with clear component separation for maintainability.
- Designed components to be minimal, interactive, and responsive.

- **Technical Choices**

- React 18 with react-scripts for simplicity and standard tooling.
- Styled-components for encapsulated styles.
- useReducer for scalable and predictable state transitions in Todo List.
- Simple structure to allow easy enhancements (routing, backend, etc.)